Spike: Task 9

Title: Game Data Structures

Author: Mohamed Shafi Uzman Fassy, 102608927

Goals / deliverables:

The goal of this spike task is to demonstrate my ability to collect and analyse software performance data while applying my knowledge in comparing and evaluating functions, IDE and complier settings.

Besides this report, what else was created?

- Code files in see C:\Users\storm\Documents\GitHub\GAMES PROGRAMMING REPO\COS30031-2023-102608927\09 - Spike - Game Data Structures\InventorySystem\InventorySystem

Technologies, Tools, and Resources used:

List of information needed by someone trying to reproduce this work.

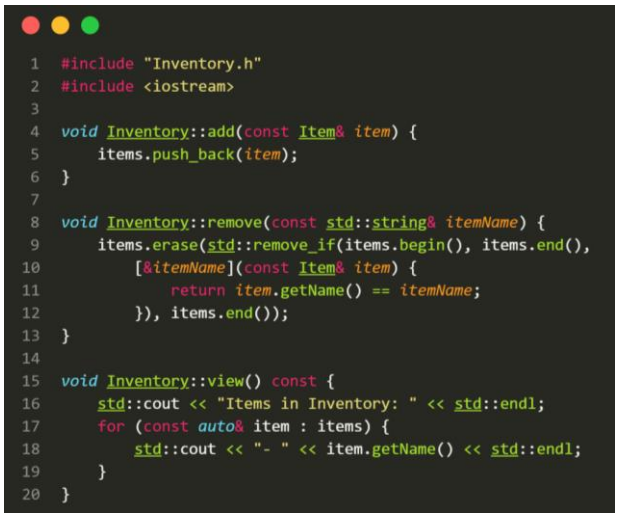
- Visual Studio 2022 Community Edition
- GitHub
- Game Specification Details Canvas Module
- OO Concepts, Patterns & Structures Canvas Modules
    - 2.1 Software Architecture of Games Lecture Slides
    - 2.2 Game States and Stages Lecture Slides
    - 3.2 Data Structures
    - 3.3 Design Patterns

Tasks undertaken:

- Evaluating Different Data Structures for Inventory Systems
- Download and install Visual Studio
- Download and install Visual Studio C++ Debugging Packages
- Create and Configure VS Project File with a .cpp file to build.
- Creating Inventory and Item Class

What we found out:

Implementing Inventory System



```cpp
#include "Inventory.h"
#include <iostream>

void Inventory::add(const Item& item) {
    items.push_back(item);
}

void Inventory::remove(const std::string& itemName) {
    items.erase(std::remove_if(items.begin(), items.end(),
        [&itemName](const Item& item) {
            return item.getName() == itemName;
        }), items.end());
}

void Inventory::view() const {
    std::cout << "Items in Inventory: " << std::endl;
    for (const auto& item : items) {
        std::cout << "- " << item.getName() << std::endl;
    }
}
```

```cpp
#pragma once
#include "Item.h"
#include <vector>
#include <algorithm>

class Inventory {
private:
    std::vector<Item> items;

public:
    void add(const Item& item);
    void remove(const std::string& itemName);
    void view() const;
};
```

The Inventory class serves as the backbone for the inventory system. As the class is fundamentally designed to store, manage, and retrieve items that a player might possess during the gameplay.

Inventory Header File : the header file for the Inventory class, we declared the methods and properties associated with the inventory. Some of the primary methods such as:

1. **addItem(Item)**: that adds an item to the inventory.
2. **removeItem(Item)**: that removes a specific item from the inventory.
3. **displayItems()**: that displays all the items currently held in the inventory.

To store the items, we use a **std::vector** data structure as this data structure was chosen due to its efficiency in appending and accessing elements, particularly if the order of items is significant.

Implementing Item

```cpp
#pragma once
#include "Item.h"
#include <vector>
#include <algorithm>

class Inventory {
private:
    std::vector<Item> items;

public:
    void add(const Item& item);
    void remove(const std::string& itemName);
    void view() const;
};

```

```cpp
#pragma once
#include <string>

class Item {
private:
    std::string name;

public:
    Item(const std::string& itemName) : name(itemName) {}
    std::string getName() const { return name; }
};

```

Item class represents individual item objects that can be added to or removed from the inventory. Each item uses an attribute such as a name to identify the item.

Item Header File: The item header file for the Item class outlines methods associated with individual items. Some fundamental methods and properties are:

1. A constructor to initialize an item with specific attributes.
2. Getter and Setter methods for each attribute.
3. Perhaps methods that dictate how the item interacts with other game elements or the environment.

Testing Outcome

Once the Inventory class and Item class has been created, we did some testing in main where we created 3 items a sword, sheild and potion in which we added them to the inventory and then displayed then, after that we remove potion from the inventory and displayed the inventory again and it shows that the potion was removed from the inventory in real time.

```cpp
#include "Inventory.h"
#include <iostream>

int main() {
    Inventory playerInventory;
    Item sword("Sword");
    Item shield("Shield");
    Item potion("Potion");

    playerInventory.add(sword);
    playerInventory.add(shield);
    playerInventory.add(potion);

    playerInventory.view();

    playerInventory.remove("Potion");
    std::cout << "\nAfter removing potion:" << std::endl;
    playerInventory.view();

    return 0;
}

```

```
Microsoft Visual Studio Debug Console
Items in Inventory:
- Sword
- Shield
- Potion

After removing potion:
Items in Inventory:
- Sword
- Shield
```